

วันที่ 3: Logic, Interactive Web & AI Assistance

ช่วงเช้า: การสร้างความโต้ตอบบนหน้าจอ (Advanced JavaScript & DOM)

- JavaScript Essentials: ทบทวนตัวแปร (Variables), เงื่อนไขตรรกะ (If-Else Conditions), และลูป (Loops เช่น `forEach`, `while`)

Data Types (ชนิดข้อมูลพื้นฐาน) :

ชนิดข้อมูล (Data Type)	ขนาด (Byte)	ช่วงข้อมูลที่เก็บได้ (Range)	คำอธิบายและข้อสังเกต
boolean / bool	1	true (1) หรือ false (0)	ใช้เก็บค่าความจริง จริงหรือเท็จ
byte	1	0 ถึง 255	เก็บตัวเลขจำนวนเต็มบวก (ไม่มีเครื่องหมายลบ)
char	1	-128 ถึง 127	เก็บอักขระ 1 ตัวอักษร (ตามรหัส ASCII)
unsigned char	1	0 ถึง 255	มีค่าและการใช้งานเหมือนกับ byte ทุกประการ
int	2	-32,768 ถึง 32,767	ตัวเลขจำนวนเต็มทั่วไป (มีทั้งค่าบวกและลบ)
unsigned int	2	0 ถึง 65,535	ตัวเลขจำนวนเต็มบวกเท่านั้น
word	2	0 ถึง 65,535	มีค่าและการใช้งานเหมือนกับ unsigned int
short	2	-32,768 ถึง 32,767	บนบอร์ด Arduino UNO จะมีค่าและการใช้งานเหมือนกับ int
long	4	-2,147,483,648 ถึง 2,147,483,647	ตัวเลขจำนวนเต็มขนาดใหญ่ (มีทั้งบวกและลบ)
unsigned long	4	0 ถึง 4,294,967,295	ตัวเลขจำนวนเต็มบวกขนาดใหญ่ นิยมใช้กับการเก็บเวลาจากฟังก์ชัน <code>millis()</code>
float	4	-3.4028235E+38 ถึง 3.4028235E+38	ตัวเลขทศนิยม (มีความแม่นยำของจุดทศนิยมประมาณ 6-7 ตำแหน่ง)
double	4	มีขนาดและช่วงข้อมูลเหมือน float	ข้อควรระวัง: บนบอร์ด UNO (AVR) double จะมีขนาด 4 Bytes เท่ากับ float ไม่ได้ใหญ่กว่าเหมือนในภาษา C ทั่วไป
char array (string)	ยืดหยุ่น	ขึ้นอยู่กับจำนวนตัวอักษร + 1	การเก็บข้อความแบบดั้งเดิมของภาษา C (<code>char str[] = "Arduino";</code>)
String (Object)	ยืดหยุ่น	ขึ้นอยู่กับหน่วยความจำ (RAM)	คลาสสำเร็จรูปสำหรับการจัดการข้อความ ใช้งานง่ายแต่กินหน่วยความจำ (SRAM) สูง

ตัวแปร (Variables) :

คำสั่ง (Keyword)	ลักษณะการใช้งาน	ตัวอย่างในงานอุตสาหกรรม
<code>const</code>	ค่าคงที่ กำหนดแล้วเปลี่ยนค่าไม่ได้ (ปลอดภัยที่สุดและป้องกันการเผลอเขียนทับ)	หมายเลขประจำเครื่อง (Machine ID), ซิดจำกัดอุณหภูมิสูงสุด
<code>let</code>	ค่าที่สามารถเปลี่ยนแปลงได้ตลอดเวลาตามสถานการณ์	สถานะการทำงานปัจจุบัน (Running/Stop), จำนวนชิ้นงานที่ผลิตได้บนสายพาน

```
// การตั้งค่าตัวแปรสำหรับเครื่องจักร
const maxTemperature = 85; // องศาเซลเซียส (ห้ามเปลี่ยน)
let currentTemperature = 45; // อุณหภูมิปัจจุบันจากเซ็นเซอร์ (เปลี่ยนแปลงได้)
let machineStatus = "RUNNING"; // สถานะเริ่มต้น
```

เงื่อนไขตรรกะ (If-Else Conditions) :

การใช้เงื่อนไขคือการสอนให้โปรแกรมรู้จักตัดสินใจตามสถานการณ์ที่เกิดขึ้น หากเงื่อนไขเป็นจริง (True) โปรแกรมจะทำงานในบล็อกแรก แต่ถ้าเป็นเท็จ (False) จะข้ามไปเช็คเงื่อนไขถัดไปหรือทำงานในบล็อก else

ตัวอย่างโค้ด :

```
let temp = 90;
if (temp > maxTemperature) {
    // ทำงานเมื่อเงื่อนไขเป็นจริง (อุณหภูมิสูงเกินขีดจำกัด)
    console.log("ALARM: แจ้งเตือนอุณหภูมิเกินกำหนด!");
    machineStatus = "STOPPED";
} else if (temp >= 70) { // ทำงานเมื่ออุณหภูมิเริ่มใกล้ถึงจุดวิกฤต
    console.log("WARNING: อุณหภูมิเริ่มสูงขึ้น ควรตรวจสอบระบบหล่อเย็น");
} else { // ทำงานเมื่อทุกอย่างอยู่ในเกณฑ์ปกติ
    console.log("NORMAL: ระบบทำงานตามปกติ");
}
```

การทำงานซ้ำ (Loops) :

ดูช่วยลดความซ้ำซ้อนในการเขียนโค้ดเมื่อต้องจัดการกับข้อมูลจำนวนมาก หรือต้องการให้โปรแกรมทำงานเดิมซ้ำๆ จนกว่าจะเสร็จสิ้นตามที่กำหนดไว้ มีด้วยกันหลายแบบดังนี้

1. While Loop

ลูปนี้จะทำงานไปเรื่อยๆ ตราบใดที่เงื่อนไขยังเป็นจริงอยู่ มักใช้กับการรอคอยสถานะบางอย่างหรือการนับจำนวนชิ้นงาน

```
let productCount = 0;
const targetProduction = 5;

// โปรแกรมจะทำงานซ้ำจนกว่าจะผลิตรอบ 5 ชิ้น
while (productCount < targetProduction) {
    productCount++; // เพิ่มค่าที่ละ 1
    console.log("เซ็นเซอร์ตรวจพบชิ้นงานที่: " + productCount);
}
```

2. For Loop

ลูปแบบ for เป็นลูปพื้นฐานที่นักพัฒนานิยมใช้มากที่สุดเมื่อ **รู้จำนวนรอบที่แน่นอนล่วงหน้าอย่างแน่นอน**

```
// ตัวอย่าง: การจำลองทดสอบแขนกล (Robotic Arm) จำนวน 3 แขนก่อนเริ่มการผลิต
console.log("--- เริ่มกระบวนการตรวจสอบแขนกล ---");

// โครงสร้าง: for (ค่าเริ่มต้น; เงื่อนไขให้ทำงานต่อ; การเปลี่ยนแปลงค่าในแต่ละรอบ)
for (let i = 1; i <= 3; i++) {
  console.log("กำลังรันโปรแกรมทดสอบแกนที่: " + i);
  // (ในสถานการณ์จริง อาจเป็นการเรียกฟังก์ชันส่งสัญญาณไปยัง Arduino หรือ PLC)
}

console.log("--- การทดสอบเสร็จสมบูรณ์ เครื่องจักรพร้อมทำงาน ---");
```

3. For Loop (Array)

ลูปนี้ใช้หมายเลขตำแหน่ง (Index) ในการเข้าถึงลำดับข้อมูล ซึ่งจะมีประโยชน์เมื่อเราต้องการอ้างอิงลำดับของข้อมูลนั้นๆ

```
// ตัวอย่าง: การตรวจสอบช่องเก็บอะไหล่ (Index เริ่มต้นที่ 0 เสมอ)
const storageBins = ["มีชิ้นงาน", "ว่าง", "ว่าง", "มีชิ้นงาน"];

for (let i = 0; i < storageBins.length; i++) {
  if (storageBins[i] === "ว่าง") {
    console.log("แจ้งเตือน: ช่องเก็บของหมายเลข " + i + " ว่าง สามารถนำชิ้นงานมาเติมได้");
  }
}
```

4. ForEach

ใช้สำหรับวนอ่านข้อมูลใน "อาร์เรย์ (Array)" ทีละตัว เหมาะอย่างยิ่งสำหรับการนำข้อมูลจาก API หรือฐานข้อมูลมาแสดงบนหน้าจอ

```
// อาร์เรย์จำลองข้อมูลเซ็นเซอร์จากจุดต่างๆ ในโรงงาน
const sensorData = [
  { location: "Zone A - ประกอบ", status: "OK" },
  { location: "Zone B - พ่นสี", status: "FAULT" },
  { location: "Zone C - บรรจุ", status: "OK" }
];

// วนลูปเพื่อเช็คสถานะและแสดงผลของแต่ละจุด
sensorData.forEach(function(sensor) {
  if (sensor.status === "FAULT") {
    console.log("พบความผิดปกติ: " + sensor.location + " โปรดส่งทีมซ่อมบำรุง");
  } else {
    console.log(sensor.location + " สถานะ: เครื่องจักรทำงานปกติ");
  }
});
```

For Loop (Array) VS ForEach :

คุณสมบัติ	for Loop	forEach Method
การเข้าถึงข้อมูล	ดึงผ่าน Index (array[i])	ดึงผ่านตัวแปร (item) ตรงๆ
ความยาวของโค้ด	ยาวกว่า (ต้องตั้งค่า i, เงื่อนไข, เพิ่มค่า)	สั้นกว่า กระชับ อ่านง่าย
การหยุดลูปรกกลางคัน	ทำได้ทันทีด้วย break	ทำไม่ได้ (ต้องใช้ลูกเล่นอื่น หรือเปลี่ยนไปใช้รูปแบบอื่น)
สถานการณ์ที่เหมาะสม	เมื่อต้องการรู้ตำแหน่งที่แน่นอน หรือต้องการค้นหาของชิ้นเดียวแล้ว หยุดการทำงานเพื่อประหยัดทรัพยากร	เมื่อต้องการประมวลผลข้อมูล "ทุกชิ้น" ในระบบ เช่น การสรุยอด การอัปเดตหน้าจอ Dashboard ทั้งหมด

- DOM Manipulation: เทคนิคการใช้ JavaScript เข้าไปเปลี่ยนแปลงข้อมูล ซ่อน/แสดงองค์ประกอบ หรือเปลี่ยนสีหน้าจอโดยไม่ต้อง Refresh หน้าเว็บ (ลดเวลาโหลดและเพิ่มความต่อเนื่องในการทำงาน)

Lab 1: getElementById (เปลี่ยนข้อความและสี)

เป้าหมายการเรียนรู้: เรียนรู้วิธีการเข้าถึง Element โดยใช้ ID และการเปลี่ยนข้อความ (innerText) รวมถึงเปลี่ยนสไตล์ (style.color)

โค้ด :

```

<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 1: Change Text</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { font-size: 20px; line-height: 1.5; font-family: sans-serif; background-color: #f4f6f9; }
    .card { margin-bottom: 20px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4"> Lab 1: JavaScript DOM Manipulation</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-primary text-white">
          1. เปลี่ยนข้อความ (Change Text)
        </div>
        <div class="card-body text-center">
          <h3>ยอดการผลิตปัจจุบัน</h3>
          <h1 id="production-count" class="display-1 text-primary fw-bold">0</h1>
          <button class="btn btn-outline-primary btn-lg w-100" onclick="updateCounter()">+ จำลองเซ็นเซอร์</button>
        </div>
      </div>
    </div>
  </div>
  <div class="text-center mt-3">
    <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
  </div>
</div>
<script>
  const countDisplay = document.getElementById("production-count");
  let currentCount = 0;
  function updateCounter() {
    currentCount++;
    countDisplay.innerText = currentCount;
  }
</script>
</body>
</html>

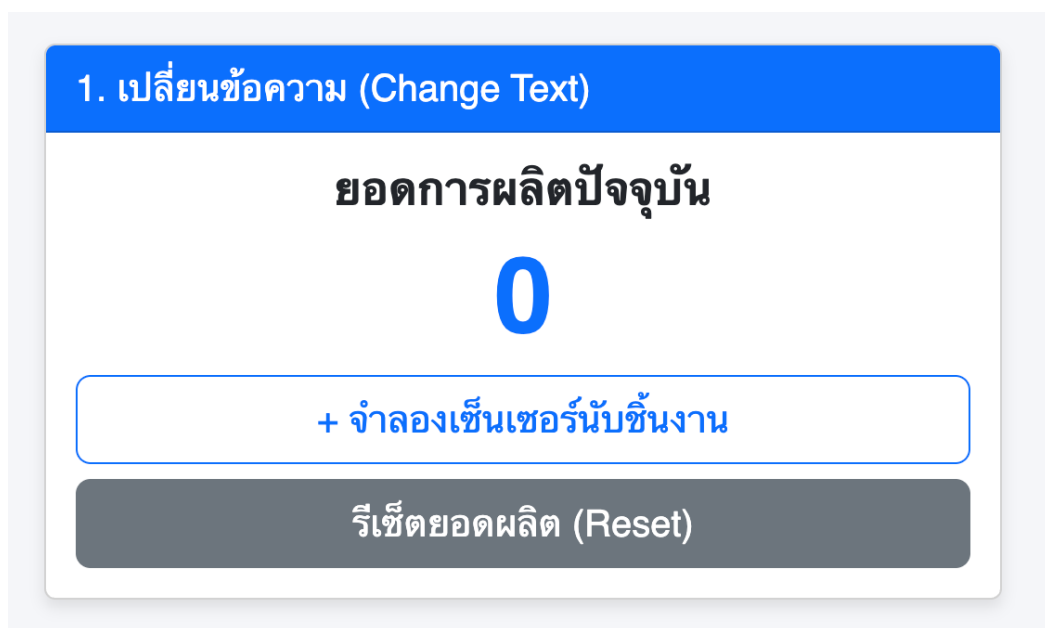
```

คำอธิบายโค้ด :

1. document.getElementById(...) คือการค้นหา HTML Element ที่มี id ตรงกับที่ระบุ
2. .innerText ใช้สำหรับอ่านหรือเขียนข้อความลงใน Element นั้น
3. .style ใช้สำหรับปรับแต่ง CSS ของ Element นั้นผ่าน JavaScript โดยตรง

โจทย์เสริม :

เพิ่มปุ่ม "Reset" เพื่อรีเซ็ตค่าตัวเลขกลับเป็น 0 ** นำ updateCounter() มาประยุกต์ใช้ **

ผลลัพธ์ :**Lab 2: getElementsByClassName (เปลี่ยนหลายค่าพร้อมกัน)**

เป้าหมายการเรียนรู้: เข้าใจการเข้าถึง Element หลายๆ ตัวพร้อมกันด้วย Class Name และการใช้ Loop (for) เพื่อจัดการข้อมูลเหล่านั้น

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 2: Change Style</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { font-size: 20px; line-height: 1.5; font-family: sans-serif; background-color: #f4f6f9; }
    .card { margin-bottom: 20px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4"> Lab 2: JavaScript DOM Manipulation</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-dark text-white">
```

```
2. เปลี่ยนสีสถานะ (Change Style)
</div>
<div class="card-body">
  <div id="machine-status" class="alert alert-success fs-4 text-center fw-bold">
    สถานะ: ปกติ (Normal)
  </div>
  <div class="d-flex gap-2">
    <button class="btn btn-danger w-50 btn-lg" onclick="triggerAlarm()">จำลองความร้อนเกิน!
  </button>
    <button class="btn btn-success w-50 btn-lg" onclick="resetStatus()">รีเซ็ตสถานะ</button>
  </div>
</div>
<div class="text-center mt-3">
  <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
</div>
</div>
</div>
<script>
  const statusBox = document.getElementById("machine-status");
  function triggerAlarm() {
    statusBox.className = "alert alert-danger fs-4 text-center fw-bold";
    statusBox.innerText = "สถานะ: แจ้งเตือนอุณหภูมิสูง! (WARNING)";
  }
  function resetStatus() {
    statusBox.className = "alert alert-success fs-4 text-center fw-bold";
    statusBox.innerText = "สถานะ: ปกติ (Normal)";
  }
</script>
</body>
</html>
```

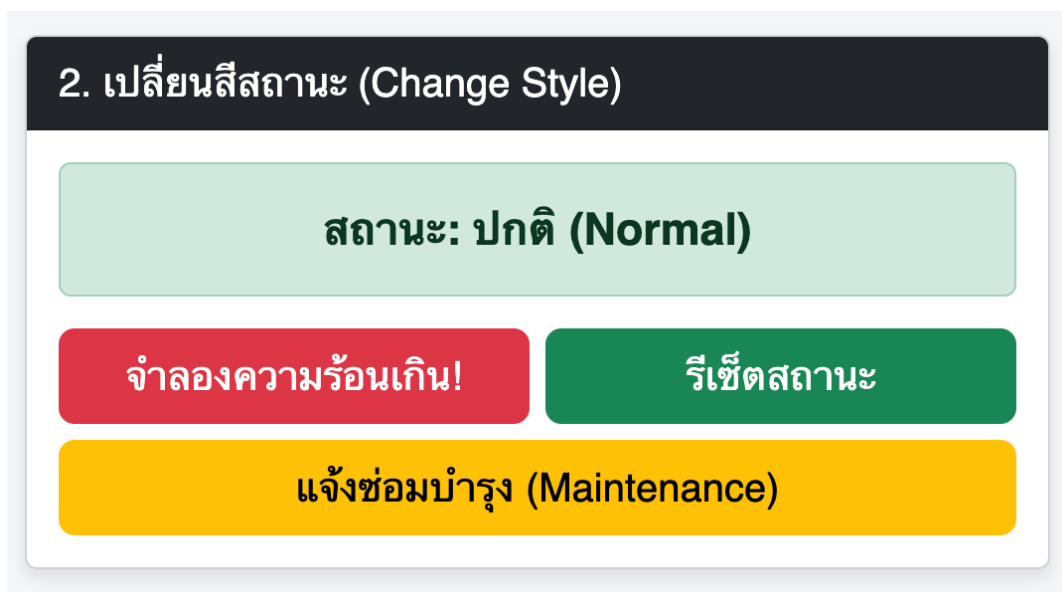
คำอธิบายโค้ด :

1. `getElementsByName()` จะคืนค่าเป็น Array-like object (รายการของ Element ทั้งหมดที่มี Class นั้น)
2. เราต้องใช้ Loop (เช่น for loop) เพื่อเข้าไปจัดการกับ Element แต่ละตัวทีละอัน
3. `.length` จะบอกให้รู้ว่า มี Element ที่ใช้ Class นี้อยู่กี่ตัว

โจทย์เสริม :

เพิ่มปุ่ม "Maintenance" เพื่อเปลี่ยนสถานะเป็น "กำลังซ่อมบำรุง" (สีเหลือง/warning) ** นำ `resetStatus()` มาเปลี่ยน class text fontstyle **

ผลลัพธ์ :



Lab 3: querySelector & querySelectorAll

เป้าหมายการเรียนรู้: เรียนรู้การค้นหาคำ Element ที่ยืดหยุ่นที่สุด โดยใช้รูปแบบเดียวกับการเขียน CSS Selectors

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 3: Hide/Show</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { font-size: 20px; line-height: 1.5; font-family: sans-serif; background-color: #f4f6f9; }
    .card { margin-bottom: 20px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4"> Lab 3: JavaScript DOM Manipulation</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-warning text-dark">
          3. ซ่อน/แสดงองค์ประกอบ (Hide/Show)
        </div>
        <div class="card-body text-center">
          <button class="btn btn-warning btn-lg w-100 mb-3" onclick="showEmergencyPanel()">⚠️ แสดงแผน
            ควบคุมฉุกเฉิน</button>

          <div id="emergency-panel" class="alert alert-danger" style="display: none;">
            <h4 class="fw-bold">🚨 แผนควบคุมฉุกเฉิน</h4>
            <p>ใช้งานเฉพาะกรณีเกิดอุบัติเหตุเท่านั้น!</p>
            <button class="btn btn-dark btn-lg" onclick="hideEmergencyPanel()">ปิดแผนควบคุม (ซ่อน)
          </div>
        </div>
      </div>
    </div>
    <div class="text-center mt-3">
      <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
    </div>
  </div>
</div>
<script>
  const emergencyPanel = document.getElementById("emergency-panel");
  function showEmergencyPanel() {
    emergencyPanel.style.display = "block";
  }
  function hideEmergencyPanel() {
    emergencyPanel.style.display = "none";
  }
</script>
</body>
</html>
```

คำอธิบายโค้ด :

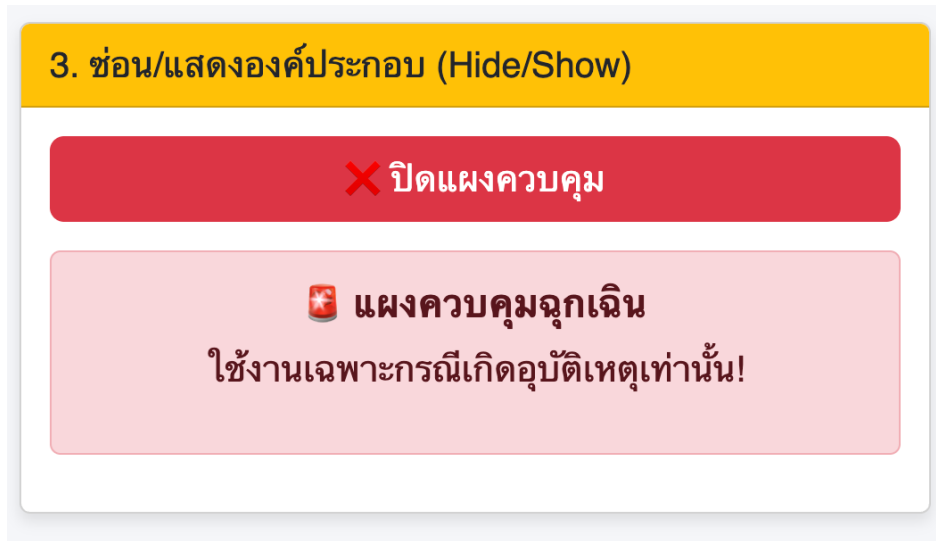
- querySelector() จะคืนค่า Element ตัวแรกที่ตรงกับเงื่อนไข CSS Selector
- querySelectorAll() จะคืนค่า NodeList ของ Element ทั้งหมดที่ตรงเงื่อนไข ซึ่งสามารถใช้ .forEach() ได้เลย
- วิธีนี้ยืดหยุ่นมาก เพราะสามารถอ้างอิงแบบซ้อนกันได้ เช่น '.container > .btn'

โจทย์เสริม :

- เปลี่ยนจากการใช้ 2 ปุ่ม (Show/Hide) ให้เหลือปุ่มเดียวที่ทำหน้าที่ "Toggle" (กดทีหนึ่งแสดง กดอีกทีซ่อน)
- เมื่อแผนควบคุมแสดงขึ้นมา ให้เปลี่ยนข้อความบนปุ่มจาก "แสดง" เป็น "ซ่อน"

** ใช้การสลับ .style.display ระหว่าง "block" กับ "none" **

ผลลัพธ์ :



Lab 4: Create & Append (สร้างและเพิ่ม Element ใหม่)

เป้าหมายการเรียนรู้: เข้าใจการสร้าง HTML Element ใหม่ด้วย JavaScript และนำไปแทรกลงในหน้าเว็บ รวมถึงการใช้ innerHTML

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 4: Create & Append</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { font-size: 20px; line-height: 1.5; font-family: sans-serif; background-color: #f4f6f9; }
    .card { margin-bottom: 20px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4"> Lab 4: JavaScript DOM Manipulation</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-info text-dark">
          4. เพิ่มข้อมูลใหม่ (Append Element)
        </div>
        <div class="card-body">
          <button class="btn btn-info btn-lg w-100 mb-3" onclick="addErrorLog()">เพิ่ม Log การแจ้งเตือน
        </button>
          <ul id="log-list" class="list-group">
          </ul>
        </div>
      </div>
      <div class="text-center mt-3">
        <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
      </div>
    </div>
  </div>
</div>
<script>
  const logList = document.getElementById("log-list");
  let logCount = 1;

  function addErrorLog() {
    const newListItem = document.createElement("li");
    // ใช้ class เดียวกับ Lab4_Ans เพื่อให้มี badge และจัดตำแหน่งสวยงาม
```



```
newListItem.className = "list-group-item list-group-item-danger d-flex justify-content-between align-items-center";

const now = new Date();
const timeStr = now.toLocaleTimeString();

// ใช้ innerHTML เพื่อใส่โครงสร้าง HTML เข้าไปในลิสต์
newListItem.innerHTML = `
  <span><strong>#${logCount}</strong> [${timeStr}] ตรวจพบความผิดปกติ</span>
  <span class="badge bg-danger rounded-pill">Error</span>
`;

logList.appendChild(newListItem);
logCount++;
}
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. .innerHTML สามารถใช้อ่านหรือ 'ใส่โครงสร้าง HTML' เข้าไปใน Element ได้เลย
2. การใช้เครื่องหมาย += หมายถึงการ 'ต่อท้าย' ข้อมูลเดิมที่มีอยู่แล้ว (Append)
3. การใช้ Backtick (` `) ช่วยให้สามารถเขียน HTML หลายบรรทัดและแทรกตัวแปร \${...} ได้ง่ายขึ้น

โจทย์เสริม :

1. เพิ่มปุ่ม "Clear Logs" เพื่อลบรายการแจ้งเตือนทั้งหมดใน (ใช้ .innerHTML = "") ** ปรับ loglist เป็นค่าว่างและ reset การนับจำนวน **

ผลลัพธ์ :

4. เพิ่มข้อมูลใหม่ (Append Element)

เพิ่ม Log การแจ้งเตือน

ล้าง

#1 [10:43:08 AM] ตรวจพบความผิดปกติ	Error
#2 [10:43:09 AM] ตรวจพบความผิดปกติ	Error
#3 [10:43:09 AM] ตรวจพบความผิดปกติ	Error

Lab 5: Disable / Enable (เปิด-ปิดการใช้งานปุ่ม)

เป้าหมายการเรียนรู้: เรียนรู้วิธีการควบคุมสถานะของฟอร์มหรือปุ่ม (Property: disabled) เพื่อป้องกันผู้ใช้งานกดซ้ำ

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 5: Disable/Enable Attribute</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { font-size: 20px; line-height: 1.5; font-family: sans-serif; background-color: #f4f6f9; }
    .card { margin-bottom: 20px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4"> Lab 5: JavaScript DOM Manipulation</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-secondary text-white">
          5. ล็อกการกดปุ่ม (Disable Button)
        </div>
        <div class="card-body text-center">
          <div class="d-flex gap-2">
            <button id="start-btn" class="btn btn-success btn-lg w-50"
onclick="startProcess()"> ▶ เริ่มสายพาน</button>
            <button id="stop-btn" class="btn btn-danger btn-lg w-50" onclick="stopProcess()"
disabled> ⛔ หยุดการทำงาน</button>
          </div>
          <div class="text-center mt-3">
            <a href="index.html" class="btn btn-secondary"> กลับหน้าหลัก</a>
          </div>
        </div>
      </div>
    </div>
  </div>
</div>
<script>
const btnStart = document.getElementById("start-btn");
const btnStop = document.getElementById("stop-btn");
function startProcess() {
  btnStart.disabled = true;
  btnStart.innerText = "⏸ กำลังทำงาน...";
  btnStop.disabled = false;
}
function stopProcess() {
  btnStart.disabled = false;
  btnStart.innerText = "▶ เริ่มสายพาน";
  btnStop.disabled = true;
}
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. .disabled เป็น Property ประเภท Boolean (true/false) ของ HTML elements จำพวก Form/Button
2. เมื่อตั้งค่าให้เป็น true ปุ่มจะกลายเป็นสีเทาและไม่สามารถคลิกได้
3. นิยมใช้เพื่อป้องกันผู้ใช้งานกดปุ่ม 'ส่งข้อมูล' ซ้ำซ้อนขณะที่ระบบกำลังประมวลผล

โจทย์เสริม :

เพิ่มปุ่ม "Emergency Stop" ที่เมื่อกดแล้วจะ Disable ทั้งปุ่ม Start และ Stop ** ใช้ attribute .disable = true; **

ผลลัพธ์ :



โจทย์เสริม 2:

เพิ่ม feature ให้ปุ่ม "Emergency Stop" สามารถกดอีกครั้งเพื่อยกเลิกสถานะ ** กำหนดสถานะ IsEmergency และใช้ ifElse **

ผลลัพธ์ :



Lab 6: classList.toggle (สลับ Class CSS)

เป้าหมายการเรียนรู้: เข้าใจการใช้ classList เพื่อจัดการ Class CSS อย่างมีประสิทธิภาพ โดยเฉพาะคำสั่ง toggle เพื่อสลับสถานะ (เช่น สลับ Dark Mode / Alert Mode)

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 6: Toggle Class</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { font-size: 20px; line-height: 1.5; font-family: sans-serif; background-color: #f4f6f9; }
    .card { margin-bottom: 20px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
    .alert-mode {
      background-color: #dc3545 !important;
      color: white !important;
      border: 5px solid #ffc107 !important;
      transition: 0.3s;
    }
    .alert-mode .card-body { color: white !important; }

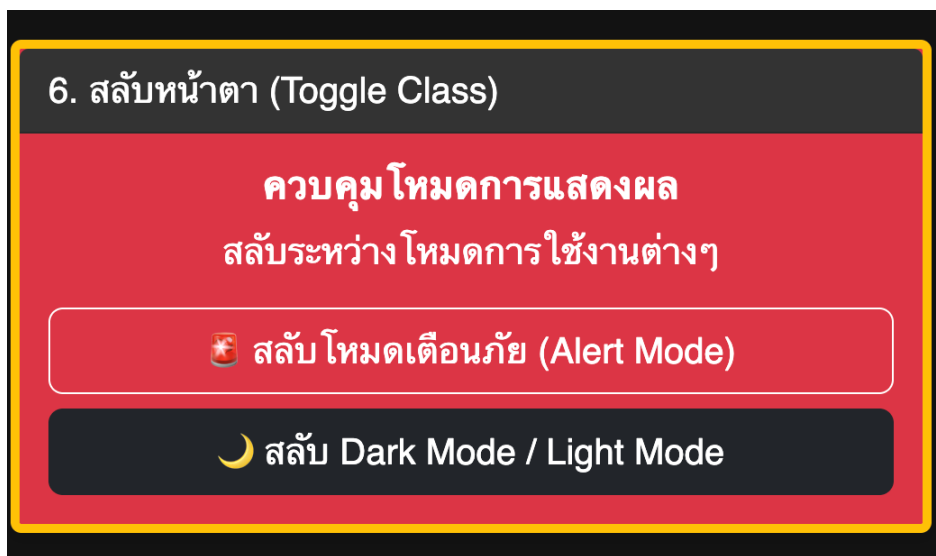
    /* ให้งุม toggle เป็นสีขาวเมื่ออยู่ใน Alert Mode */
    .alert-mode .btn-outline-danger {
      color: white !important;
      border-color: white !important;
    }
    /* เมื่อ hover ใน Alert Mode ให้พื้นหลังงุมเป็นสีขาวและตัวหนังสือสีแดง */
    .alert-mode .btn-outline-danger:hover {
      background-color: white !important;
      color: #dc3545 !important;
    }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">🏭 Lab 6: JavaScript DOM Manipulation</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card" id="factory-dashboard">
        <div class="card-header">
          6. สลับหน้าตา (Toggle Class)
        </div>
        <div class="card-body text-center">
          <h4>หน้าตาสามารถเปลี่ยนโหมดได้</h4>
          <p>ใช้ ClassList เพื่อเปลี่ยน CSS ทั้งชุด</p>
          <button class="btn btn-outline-danger btn-lg mt-2" onclick="toggleAlert()">🔴 สลับโหมดเตือนภัย
(Toggle)</button>
        </div>
      </div>
      <div class="text-center mt-3">
        <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
      </div>
    </div>
  </div>
</div>
<script>
const dashboard = document.getElementById("factory-dashboard");
function toggleAlert() {
  dashboard.classList.toggle("alert-mode");
}
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. .classList เป็น Property ที่ใช้จัดการกับ class ทั้งหมดของ Element นั้นๆ
2. .toggle() จะทำหน้าที่ 'สลับ': ถ้ายังไม่มี class นี้จะทำการเพิ่ม (Add) แต่ถ้ามีอยู่แล้วจะทำการลบ (Remove)
3. .contains() ใช้ตรวจสอบว่า Element นั้นมี class ที่ระบุอยู่หรือไม่ (คืนค่าเป็น true/false)

โจทย์เสริม :

1. สร้าง Class ใหม่ใน CSS ชื่อ .dark-mode (พื้นหลังดำ ตัวอักษรขาว) **** เพิ่ม class css ****
2. เพิ่มปุ่มเพื่อ Toggle ระหว่างโหมดปกติและ Dark Mode สำหรับตัว Card **** ประยุกต์ใช้ toggleAlert() และเจาะจงไปที่ body ****

ผลลัพธ์ :**Lab 7: Read Input Value (ดึงค่าจากช่องกรอกข้อมูล)**

เป้าหมายการเรียนรู้: เรียนรู้การอ่านค่า (Read) และการกำหนดค่า (Write) ให้กับ HTML Input รวมถึงการตรวจสอบความถูกต้องของข้อมูล (Validation)

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Lab 7: Read Input Value</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4"> Lab 7: Read Input Value</h1>
  <div class="row justify-content-center">
    <div class="col-md-8">
      <div class="card">
        <div class="card-header bg-dark text-white">
          7. ดึงค่าจากช่องพิมพ์ (Read Input Value)
        </div>
        <div class="card-body">
          <!-- ส่วนตั้งค่านาตาบรรทัด -->
          <div class="row mb-3 bg-light p-3 rounded border">
```

หลักสูตรที่ 1: Advanced Front-end & UI/UX Design

```
// คำนวณ พาเลต, โฟล, และขวดที่เหลือ
let bottlesPerPallet = bottlesPerpacksize * packsizesPerPallet;
let pallets = Math.floor(totalBottles / bottlesPerPallet);
let remainderAfterPallets = totalBottles % bottlesPerPallet;

let packsizes = Math.floor(remainderAfterPallets / bottlesPerpacksize);
let bottles = Math.round(remainderAfterPallets % bottlesPerpacksize);

// แสดงผลลัพธ์
displaySpeed.innerHTML = `
  <div class="fs-5 text-primary">
    <strong>ผลการคำนวณ :</strong> ${pallets} พาเลต, ${packsizes} แพ็ค, ${bottles} ขวด
    <br><small class="text-muted">(รวมทั้งหมด ${totalBottles} ขวด)</small>
  </div>
`;
displaySpeed.className = "alert alert-success fs-4 mb-0";
}

// เรียกครั้งแรกเพื่อให้แสดงผลทันที
updateQty();
</script>
</body>
</html>
```

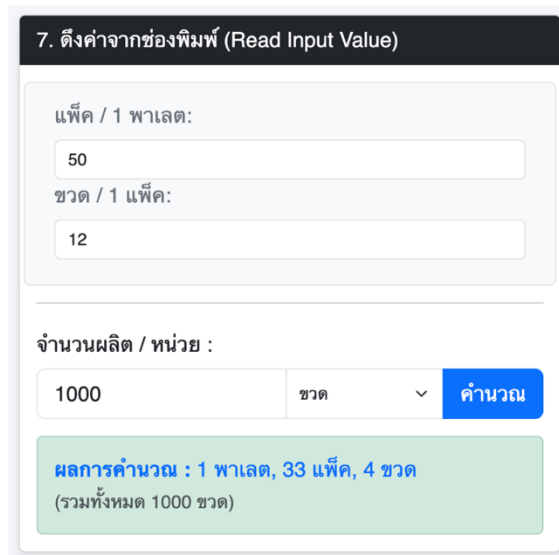
คำอธิบายโค้ด :

1. `.value` ใช้เพื่อดึง 'ข้อมูลที่ใช้พิมพ์' ลงในช่อง `<input>` หรือ `<textarea>`
2. `isNaN()` เป็นฟังก์ชันสำหรับตรวจสอบว่าค่าที่รับมา 'ไม่ใช่ตัวเลข' (Not a Number) หรือไม่
3. ค่าที่ได้จาก `.value` มักจะเป็น String เสมอ แม้ว่า input type จะเป็น number ก็ตาม

โจทย์เสริม :

เพิ่ม feature ในการเปลี่ยนค่ากำหนด พาเลตและ แพ็คให้สามารถเปลี่ยนแปลงค่าจำนวนได้อัตโนมัติ ** ใช้ `oninput=""` **

ผลลัพธ์ :



- Event Listeners: การดักจับพฤติกรรมผู้ใช้งาน เช่น การคลิกปุ่ม, การพิมพ์ข้อความลงช่อง, หรือการเปลี่ยนค่าใน Dropdown เพื่อสั่งให้โปรแกรมทำงานทันที

Lab 1: Click Event (ดักจับการคลิก)

เป้าหมายการเรียนรู้ : เข้าใจการดักจับเหตุการณ์คลิกปุ่ม และการใช้ JavaScript เพื่อเปลี่ยน CSS (style.display) และเปลี่ยนคลาสของปุ่ม

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Lab 1: Click Event</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>body { background-color: #f4f6f9; font-size: 20px; }</style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">Event Lab 1: Click Event</h1>
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card shadow-sm">
        <div class="card-header bg-primary text-white">1. ดักจับการคลิก (Click)</div>
        <div class="card-body text-center">
          <button id="check-btn" class="btn btn-primary btn-lg w-100">ตรวจสอบสถานะเครื่องจักร</button>

          <div id="status-panel" class="alert alert-info mt-4 fs-4" style="display: none;">
            สถานะ: เครื่องจักรกำลังทำงานตามปกติ
          </div>
        </div>
      </div>
      <div class="text-center mt-3">
        <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
      </div>
    </div>
  </div>
</div>
<script>
  // 1. อ้างอิงถึง Element ในหน้าจ
  const checkBtn = document.getElementById("check-btn");
  const statusPanel = document.getElementById("status-panel");

  /*
  ส่วนเสริมความเข้าใจ:
  addEventListener() ทำหน้าที่ "ดักฟัง" หรือ "รอรับ" เหตุการณ์ที่เกิดขึ้นกับ Element นั้นๆ
  โดยรับค่า 2 อย่างคือ:
  1. ชื่อเหตุการณ์ (เช่น "click")
  2. ฟังก์ชันที่จะให้ทำงานเมื่อเกิดเหตุการณ์นั้น
  */

  // 2. สั่งให้ปุ่ม รอรับการ "click"
  checkBtn.addEventListener("click", function() {
    // เมื่อปุ่มถูกคลิก จะเข้ามาทำงานในบล็อกนี้

    // สั่งเปลี่ยน CSS เพื่อให้รอบแสดงผลปรากฏขึ้น
    statusPanel.style.display = "block";

    // เปลี่ยนข้อความบนปุ่ม เพื่อให้รู้ว่ากดไปแล้ว
    checkBtn.innerText = "ตรวจสอบเรียบร้อยแล้ว!";
    // ลบความเป็นปุ่มและเปลี่ยนสี
    checkBtn.className = "btn btn-success btn-lg w-100 disabled";
  });
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. ใช้ `addEventListener('click', ...)` เพื่อสั่งให้รอดักจับตอนที่ปุ่มถูกคลิก
2. `style.display = 'block'` คือการทำให้ element ที่ซ่อนอยู่ปรากฏขึ้นมา
3. สามารถเปลี่ยนข้อความด้วย `innerText` และเปลี่ยนสีด้วย `className`

โจทย์เสริม :

เพิ่มปุ่มปิดการแสดงผล ** ใช้ `.addEventListener("ชื่อเหตุการณ์", function() { ... })` **

ผลลัพธ์ :

1. ดักจับการคลิก (Click)

 ตรวจสอบสถานะเครื่องจักร

 สถานะ: เครื่องจักรกำลังทำงานตามปกติ

 ปิดการแสดงผล

Lab 2: Input Event (ดักจับการพิมพ์)

เป้าหมายการเรียนรู้ : เข้าใจการทำงานของ Event 'input' แบบ Real-time เพื่อประเมินค่าความยาวของข้อความ (Validate) โดยไม่ต้องกดส่งโค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Lab 2: Input Event</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>body { background-color: #f4f6f9; font-size: 20px; }</style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">🔧 Event Lab 2: Input Event</h1>
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card shadow-sm">
        <div class="card-header bg-warning text-dark">2. ดักจับการพิมพ์ข้อความ (Input)</div>
        <div class="card-body">
          <label class="form-label fw-bold">กรอกรหัสพนักงาน:</label>
          <input type="text" id="emp-input" class="form-control form-control-lg" placeholder="เช่น EMP-01">
          <div id="feedback" class="form-text text-danger mt-2" style="min-height: 24px;"></div>
        </div>
        <div class="text-center mt-3">
          <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
        </div>
      </div>
    </div>
  </div>
</div>
<script>
  // 1. อ้างอิงถึง Element ในหน้าจ
  const empInput = document.getElementById("emp-input");
```

```
const feedback = document.getElementById("feedback");

/*
 ส่วนเสริมความเข้าใจ:
Event "input" จะทำงานแบบ Real-time ทุกครั้งที่มีการพิมพ์หรือลบข้อความในช่อง Input
เหมาะกับการทำระบบ Validate ข้อมูลทันทีโดยไม่ต้องรอให้ผู้ใช้กดปุ่ม Submit
*/

// 2. ดักจับเหตุการณ์ "input" ทันทีที่มีการพิมพ์
empInput.addEventListener("input", function() {
  // ดึงค่าปัจจุบันที่อยู่ในช่อง Input
  let val = empInput.value;

  if (val.length === 0) {
    // ถ้าลบจนหมด ให้ล้างข้อความทิ้ง
    feedback.innerText = "";
    empInput.style.borderColor = "";
  } else if (val.length < 6) {
    // ถ้าน้อยกว่า 6 ตัวอักษร ให้แจ้งเตือนสีแดง
    feedback.innerText = "❌ รหัสพนักงานต้องอย่างน้อย 6 ตัวอักษร";
    feedback.className = "form-text text-danger mt-2 fw-bold";
    empInput.style.borderColor = "red";
  } else {
    // ถ้าครบ 6 ตัวอักษรขึ้นไป ให้ขึ้นสีเขียวผ่าน
    feedback.innerText = "✅ รหัสถูกต้องพร้อมใช้งาน";
    feedback.className = "form-text text-success mt-2 fw-bold";
    empInput.style.borderColor = "green";
  }
});
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. 'input' จะถูกเรียกทุกครั้งที่มีการพิมพ์ตัวอักษรลงไปในช่วงกรอก
2. .value ใช้เพื่อดึงข้อความปัจจุบัน และ .length ใช้เพื่อหาความยาวของข้อความ
3. ใช้ if-else เพื่อกำหนดการแสดงผลและสีขอบ

โจทย์เสริม :

เพิ่มให้แสดงผลเมื่อพิมพ์อักษรเกิน 10 ตัวและไม่ให้พิมพ์ต่อ **** .value = val.substring(,)** เพื่อจำกัดการพิมพ์**

ผลลัพธ์ :

2. ดักจับการพิมพ์ (Input)

กรอกรหัสพนักงาน (4-10 ตัว):

✅ รหัสถูกต้องพร้อมใช้งาน

Lab 3: Change Event (ดักจับการเปลี่ยนตัวเลือก)

เป้าหมายการเรียนรู้ : เรียนรู้การใช้ Event 'change' ซึ่งมักใช้กับ Dropdown (<select>) เพื่อดึงค่า Value ล่าสุดที่ผู้ใช้เลือกไว้ :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Lab 3: Change Event</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>body { background-color: #f4f6f9; font-size: 20px; }</style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">📌 Event Lab 3: Change Event</h1>
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card shadow-sm">
        <div class="card-header bg-success text-white">3. ดักจับการเปลี่ยนค่า (Change)</div>
        <div class="card-body text-center">
          <label class="form-label fw-bold d-block text-start">เลือกโหมดการทำงาน:</label>
          <select id="mode-select" class="form-select form-select-lg mb-3">
            <option value="">-- กรุณาเลือกโหมด --</option>
            <option value="slow">🐢 โหมดประหยัดพลังงาน (Slow)</option>
            <option value="normal">🟢 โหมดปกติ (Normal)</option>
          </select>

          <div id="mode-display" class="alert alert-secondary fs-4 mb-0">
            สถานะปัจจุบัน: ยังไม่ได้เลือก
          </div>
        </div>
      </div>
      <div class="text-center mt-3">
        <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
      </div>
    </div>
  </div>
</div>
<script>
  // 1. อ้างอิงถึง Element ในหน้าจอ
  const modeSelect = document.getElementById("mode-select");
  const modeDisplay = document.getElementById("mode-display");

  /*
  📌 ส่วนเสริมความเข้าใจ:
  Event "change" จะทำงานก็ต่อเมื่อมีการ "เปลี่ยนตัวเลือก" และ "คลิกเลือกเสร็จแล้ว"
  เหมาะกับพวก Dropdown (<select>), Checkbox หรือ Radio Button
  */

  // 2. ดักจับเหตุการณ์ "change"
  modeSelect.addEventListener("change", function() {
    // ดึงค่าที่ผู้ใช้เลือก (value ของ <option>)
    let mode = modeSelect.value;

    // ใช้ if-else เพื่อกำหนดข้อความและสีที่แตกต่างกันตามโหมด
    if (mode === "slow") {
      modeDisplay.innerHTML = "🐢 <strong>กำลังทำงาน:</strong> โหมดประหยัดพลังงาน";
      modeDisplay.className = "alert alert-warning fs-4 mb-0";
    } else if (mode === "normal") {
      modeDisplay.innerHTML = "🟢 <strong>กำลังทำงาน:</strong> โหมดปกติ";
      modeDisplay.className = "alert alert-success fs-4 mb-0";
    } else {
      // กรณีเลือกกลับไป "-- กรุณาเลือกโหมด --"
      modeDisplay.innerHTML = "สถานะปัจจุบัน: ยังไม่ได้เลือก";
      modeDisplay.className = "alert alert-secondary fs-4 mb-0";
    }
  });
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. 'change' จะถูกเรียกเมื่อมีการเลือก Dropdown ใหม่และคลิกตัวเลือกเสร็จสิ้น
2. ดึงค่าของ option ที่เลือกด้วย .value
3. ตัวอย่างนี้มีการเปลี่ยนสีของทั้งหน้าจอกด้วย document.body.style

โจทย์เสริม :

เพิ่มโหมดเร่งด้วยเป็นสีส้มและเพิ่ม feature ให้เปลี่ยนสีพื้นหลัง body ตามโหมดนั้นๆ

ผลลัพธ์ :

3. ดักจับการเปลี่ยนค่า (Change)

เลือกโหมดการทำงาน:



โหมดเร่งด่วน (Fast)



กำลังทำงาน: โหมดเร่งด่วน!

Lab 4: Keyboard Event (ดักจับแป้นพิมพ์)

เป้าหมายการเรียนรู้ : เข้าใจการดักจับการกดคีย์บอร์ดด้วย 'keyup' และตรวจจับปุ่มพิเศษ เช่น ปุ่ม Enter

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Lab 4: Keyboard Event</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>body { background-color: #f4f6f9; font-size: 20px; }</style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">■ Event Lab 4: Keyboard Event</h1>
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card shadow-sm border-info">
        <div class="card-header bg-info text-dark">4. ดักจับปุ่มกด (Keyboard)</div>
        <div class="card-body">
          <label class="form-label fw-bold">พิมพ์หมายเหตุ (จำกัด 50 ตัวอักษร):</label>
          <textarea id="note-input" class="form-control" rows="3" placeholder="ลองพิมพ์อะไรบางอย่าง
..."></textarea>

          <div class="d-flex justify-content-between mt-2">
            <small id="char-count" class="text-muted">จำนวนตัวอักษร: 0/50</small>
            <small id="key-info" class="badge bg-secondary">ปุ่มล่าสุด: -</small>
          </div>
        </div>
      </div>
      <div class="text-center mt-3">
        <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
      </div>
    </div>
  </div>
</div>
<script>
  const noteInput = document.getElementById("note-input");
  const charCount = document.getElementById("char-count");
  const keyInfo = document.getElementById("key-info");

  /*
  📌 ส่วนเสริมความเข้าใจ:
  - "keydown": ทำงานตอนกดปุ่มลงไป
  - "keyup": ทำงานตอนปล่อยปุ่ม (นิยมใช้เพราะค่าใน input จะอัปเดตเรียบร้อยแล้ว)
  - "e.key": ใช้เช็คว่าคุณกดปุ่มอะไร (เช่น Enter, Shift, a, b, c)
  */

  // 🎯 โจทย์เสริม: ลองเช็คว่าถ้าผู้ใช้กดปุ่ม "Enter" ให้แสดง Alert ว่า "คุณกดปุ่ม Enter!"

  noteInput.addEventListener("keyup", function(event) {
    // 1. นับจำนวนตัวอักษร
    let length = noteInput.value.length;
    charCount.innerText = `จำนวนตัวอักษร: ${length}/50`;

    // 2. แสดงชื่อปุ่มที่เพิ่งกด
    keyInfo.innerText = `ปุ่มล่าสุด: ${event.key}`;

    // 3. เปลี่ยนสีถ้ายาวเกิน 50
    if (length > 50) {
      charCount.className = "text-danger fw-bold";
      noteInput.style.borderColor = "red";
    } else {
      charCount.className = "text-muted";
      noteInput.style.borderColor = "";
    }

    // 4. ตรวจจับปุ่ม Enter (เฉลยโจทย์เสริม)
    if (event.key === "Enter") {
      console.log("มีการกดปุ่ม Enter");
      // ปรับสี badge ชั่วคราวเพื่อให้รู้ว่าตรวจจับได้
      keyInfo.className = "badge bg-success";
      setTimeout(() => keyInfo.className = "badge bg-secondary", 500);
    }
  })
}
```

```
});  
</script>  
</body>  
</html>
```

คำอธิบายโค้ด :

1. 'keyup' จะทำงานเมื่อผู้ใช้กดปุ่มบนคีย์บอร์ดแล้วปล่อยมือ
2. เราสามารถรับพารามิเตอร์ (เช่น event) เพื่อตรวจสอบว่าผู้ใช้กดปุ่มอะไร ด้วยคำสั่ง event.key

โจทย์เสริม :

เพิ่ม feature ชิดจำกัดการพิมพ์เหมือน Lab2 cละเพิ่มปุ่มล้างข้อความ ****สร้าง clearBtn.addEventListener("click", function() {});****

ผลลัพธ์ :

4. ดักจับปุ่มกด (Keyboard)

พิมพ์หมายเหตุ (จำกัด 50 ตัวอักษร):

จำนวนตัวอักษร: 0/50 ปุ่มล่าสุด: -

 **ล้างข้อความทั้งหมด**

Lab 5: Mouse Event (ดักจับเมาส์ Hover)

เป้าหมายการเรียนรู้ : เรียนรู้การใช้ mouseenter และ mouseleave เพื่อสร้าง Hover Effect (เมื่อชี้เมาส์และเมื่อเอาเมาส์ออก)

โค้ด :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Lab 5: Mouse Event</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>
    body { background-color: #f4f6f9; font-size: 20px; }
    #hover-box {
      height: 200px;
      transition: 0.3s;
      cursor: pointer;
      display: flex;
      align-items: center;
      justify-content: center;
      border: 3px dashed #ccc;
      border-radius: 15px;
      background-color: white;
    }
    .highlight {
      background-color: #e3f2fd !important;
      border-color: #2196f3 !important;
      transform: scale(1.02);
    }
  </style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">👋 Event Lab 5: Mouse Event</h1>
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card shadow-sm">
        <div class="card-header bg-dark text-white">5. ดักจับการใช้เมาส์ (Mouse Hover)</div>
        <div class="card-body text-center">
          <p>ลองเอาเมาส์มาวางในกรอบด้านล่าง</p>

          <div id="hover-box">
            <h2 id="box-text">เมาส์อยู่นอกกรอบ</h2>
          </div>

          <p id="hint" class="mt-3 text-muted fs-6">mouseenter = เมาส์เข้า | mouseleave = เมาส์ออก</p>
        </div>
        <div class="text-center mt-3">
          <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
        </div>
      </div>
    </div>
  </div>
</div>
<script>
  const hoverBox = document.getElementById("hover-box");
  const boxText = document.getElementById("box-text");

  /* 📌 ส่วนเสริมความเข้าใจ:
  - "mouseenter": ทำงานเมื่อเมาส์ "เคลื่อนเข้าไป" ในขอบเขตของ Element
  - "mouseleave": ทำงานเมื่อเมาส์ "เคลื่อนออกไป" จากขอบเขตของ Element
  เหมาะกับการทำ Effect ตอบสนอง เช่น ปุ่มขยายใหญ่ขึ้น หรือแสดงคำอธิบายเพิ่มเติม
  */

  // 🌟 โทษเสริม: เมื่อเมาส์เข้ามา ให้เปลี่ยนสีตัวหนังสือเป็นสีน้ำเงิน และเมื่อออกให้กลับเป็นสีดำ

  // 1. ดักจับตอนเมาส์เข้า
  hoverBox.addEventListener("mouseenter", function() {
    boxText.innerText = "🌟 เมาส์เข้ามาแล้ว!";
    boxText.style.color = "#0d6efd"; // สีน้ำเงิน
    hoverBox.classList.add("highlight"); // เพิ่ม Class CSS เพื่อความสวยงาม
  });

  // 2. ดักจับตอนเมาส์ออก
```

```
hoverBox.addEventListener("mouseleave", function() {  
    boxText.innerText = "เมาส์อยู่นอกกรอบ";  
    boxText.style.color = "black";  
    hoverBox.classList.remove("highlight"); // เอา Class ออก  
});  
</script>  
</body>  
</html>
```

คำอธิบายโค้ด :

1. 'mouseenter' เปรียบเสมือน CSS :hover แต่สามารถเขียน Logic ต่อได้หลากหลายกว่า
2. 'mouseleave' ใช้จัดการเพื่อรีเซ็ตค่าต่างๆ กลับเป็นค่าเดิมเมื่อเมาส์ถูกเลื่อนออกไป

โจทย์เสริม :

เพิ่ม feature เมื่อเอาเมาส์ไปชี้ให้เปลี่ยนจากเส้นปะเป็นเส้นทึบ **`** .style.borderStyle = "solid"; **`**

ผลลัพธ์ :

5. ดักจับเมาส์ (Mouse Hover)

 **เมาส์เข้ามาแล้ว!**

Lab 6: Form Submit Event (ดักจับการส่งฟอร์ม)

เป้าหมายการเรียนรู้ : เข้าใจการหยุดพฤติกรรมดั้งเดิมของ Browser ด้วย preventDefault() เพื่อไม่ให้หน้าจอ Refresh ตอนกดปุ่ม Submit ไปได้ :

```
<!DOCTYPE html>
<html lang="th">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Lab 6: Form Event</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
  <style>body { background-color: #f4f6f9; font-size: 20px; }</style>
</head>
<body>
<div class="container py-4">
  <h1 class="text-center mb-4">📌 Event Lab 6: Form Event</h1>
  <div class="row justify-content-center">
    <div class="col-md-6">
      <div class="card shadow-sm border-danger">
        <div class="card-header bg-danger text-white">6. การส่งฟอร์ม (Form Submit)</div>
        <div class="card-body">
          <form id="report-form">
            <div class="mb-3">
              <label class="form-label">ชื่อผู้แจ้ง:</label>
              <input type="text" id="reporter-name" class="form-control" required>
            </div>
            <div class="mb-3">
              <label class="form-label">รายละเอียดปัญหา:</label>
              <textarea id="report-desc" class="form-control" required></textarea>
            </div>
            <button type="submit" class="btn btn-danger w-100">ส่งรายงานความผิดปกติ</button>
          </form>

          <div id="result-alert" class="alert alert-success mt-4" style="display: none;">
            <strong>สำเร็จ!</strong> ข้อมูลของคุณถูกส่งแล้วโดยไม่ Refresh หน้าจอ
          </div>
        </div>
      </div>
      <div class="text-center mt-3">
        <a href="index.html" class="btn btn-secondary">กลับหน้าหลัก</a>
      </div>
    </div>
  </div>
</div>
<script>
  const reportForm = document.getElementById("report-form");
  const resultAlert = document.getElementById("result-alert");

  /*
  📌 ส่วนเสริมความเข้าใจ:
  - Event "submit" จะทำงานเมื่อมีการกดปุ่ม Submit ในฟอร์ม
  - สำคัญมาก: ปกติฟอร์มจะส่งและโหลดหน้าใหม่ (Refresh) อัตโนมัติ
  - เราต้องใช้ "event.preventDefault()" เพื่อหยุดการโหลดหน้าใหม่
  เพื่อให้เราประมวลผลข้อมูลด้วย JavaScript ต่อได้เอง (เช่น ส่งผ่าน API)
  */

  // 🚫 ใจเย็น! ลองดึงค่าจาก reporter-name มาแสดงใน Alert แจ้งเตือนหลังจากกดปุ่ม
  reportForm.addEventListener("submit", function(event) {
    // 1. ป้องกันไม่ให้หน้าเว็บรีเฟรช (สำคัญที่สุด!)
    event.preventDefault();

    // 2. ดึงข้อมูลจากฟอร์ม
    const name = document.getElementById("reporter-name").value;
    const desc = document.getElementById("report-desc").value;

    console.log("ส่งข้อมูล:", { name, desc });

    // 3. แสดงผลลัพธ์บนหน้าจอ
    resultAlert.style.display = "block";
    resultAlert.innerHTML = `<strong>ขอบคุณคุณ ${name}!</strong> รายงานของคุณคือ: ${desc}`;

    // 4. ล้างข้อมูลในฟอร์ม (Reset)
    reportForm.reset();
  });
</script>
</body>
</html>
```

คำอธิบายโค้ด :

1. การ Submit รูปแบบปกติจะทำให้เว็บโหลดหน้าใหม่ ซึ่งทำให้ข้อมูล JavaScript ถูกรีเซ็ตทั้งหมด
2. เราจึงต้องเรียกใช้ event.preventDefault() แทนที่เป็นบรรทัดแรก
3. .reset() เป็นฟังก์ชันสำหรับล้างค่าทั้งหมดในฟอร์มโดยอัตโนมัติ

โจทย์เสริม :

เพิ่ม feature หน่วงเวลา 1 วินาทีเพื่อเป็นชะลอการใช้งานของ user เพื่อเผื่อเวลาในการส่งข้อมูล **** setTimeout(() => {}, มิลลิวินาที); ****

ผลลัพธ์ :

6. การส่งฟอร์ม (Form Submit)

ชื่อผู้แจ้ง:

รายละเอียดปัญหา:

ส่งรายงานความผิดปกติ

ขอบคุณคุณ นาย A! ส่งรายงานแล้ว: ไลน์ L-01
เครื่องจักรมีปัญหา

ช่วงบ่าย: การเขียนโปรแกรมร่วมกับ AI (AI-Assisted Development)

- Client-side Form Validation: การใช้ AI ช่วยเขียนฟังก์ชัน JavaScript สำหรับตรวจสอบความถูกต้องของข้อมูลก่อนส่งเข้าเซิร์ฟเวอร์ เช่น ห้ามเว้นว่าง, ต้องเป็นตัวเลขเท่านั้น, หรือรูปแบบวันที่ต้องถูกต้อง ป้องกันขยะเข้าสู่ระบบ โดยตรวจสอบด้วยการให้ ส่งไฟล์ json callapi มาที่ server ถ้าถูกขึ้นหน้าจอแสดงผลมาให้ดูเป็น dashboard ที่เครื่องเรา

Workshop 1

API Workshop: Plant Lookup

เชื่อมต่อไปยังเครื่อง Server (ngrok)

Client_id	1-20
plant_code	ใส่ 4 หลัก ค้นหา
Plant_name	ชื่อโรงงานจะขึ้นที่นี่...

Send to Server

แหล่งที่มาของโค้ด :

คำอธิบาย: เมื่อกรอกรหัสโรงงานครบ 4 หลัก ระบบจะเรียก API ไปดึงชื่อโรงงานมาแสดงอัตโนมัติ และเมื่อกดปุ่ม "Send to Server" จะเป็นการส่งข้อมูลทั้งหมดไปบันทึกที่ Server

ผลลัพธ์: ได้ชื่อโรงงานที่ถูกต้องแสดงบนหน้าจอ และระบบจะส่งสถานะการทำงานไปแจ้งเตือนที่ Dashboard บน Server ทันที

Workshop 2

Plant Search (Dropdown)

เรียกใช้งานข้อมูลผ่าน API

Client_id

ระบุ Client ID (1-20)

plant_code

3201

▼

Search

Plant_name

📍 ปทุมธานี

Send to Server

Plants loaded successfully

คำอธิบาย: ระบบจะดึงรายชื่อรหัสโรงงานทั้งหมดจาก API มาสร้างเป็น Dropdown ให้เลือกโดยอัตโนมัติเมื่อเปิดหน้าเว็บ เมื่อผู้ใช้เลือกตัวเลือกหรือกดปุ่ม Search ระบบจะดึงชื่อโรงงานภาษาไทยมาแสดง และเมื่อกดปุ่ม "Send to Server" จะส่งข้อมูล Client ID และชื่อโรงงานไปบันทึกที่ Server

ผลลัพธ์: ผู้ใช้เลือกโรงงานได้สะดวกขึ้นจากรายการที่มีให้ ชื่อโรงงานที่ถูกต้องจะปรากฏบนหน้าจอ และระบบจะส่งสัญญาณแจ้งเตือนไปยัง Dashboard บน Server ทั้งตอนที่ค้นหาเจอและตอนที่กดส่งข้อมูลสำเร็จครับ

Workshop 3

Plant Search (Dropdown)

เรียกใช้งานข้อมูลผ่าน API

Client_id

ระบุ Client ID (1-20)

plant_id

3201

▼

Search

Plant_name

ปทุมธานี

line_id

1

▼

Send to Server

Lines loaded for plant 3201

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบจะดึงรายชื่อโรงงานมาใส่ใน Dropdown แรกอัตโนมัติ และเมื่อผู้ใช้เลือกโรงงาน ระบบจะทำงาน 2 อย่างพร้อมกันคือ ดึงชื่อโรงงานมาแสดง และไปดึงรายชื่อ Line ID ที่เกี่ยวข้องกับโรงงานนั้นมาใส่ใน Dropdown ที่สองเพื่อให้เลือกข้อมูลได้ครบถ้วนก่อนส่ง

ผลลัพธ์: ผู้ใช้สามารถเลือกข้อมูลแบบสัมพันธ์กัน (Cascading Dropdown) ได้อย่างถูกต้อง และเมื่อกดส่ง ข้อมูลทั้ง Client ID, ชื่อโรงงาน และ Line ID จะถูกส่งไปบันทึกพร้อมแจ้งเตือนที่ Dashboard บน Server ครับ

Workshop 4

Plant & Line Selection

เลือกข้อมูลเพื่อดูรายชื่อ SKU ในตาราง

Client_id

plant_code
 ▼ Search

Plant_name

line_id
 ▼

Send to Server

รายชื่อ SKU ที่พบ

SKU ID	รายละเอียดสินค้า (Description & Package)
1360	100Plus Citrus Can325ml 24NR (Export) 325ml x 24
1119	Chakulza GT Honey Lemon 320ml 1x24NR 320ml x 24
1128	Chakulza GT Honey Lemon 320ml 6x4NR(OEM) 320ml x 24
1282	Chakulza GT Kyoho Grape 320ml 1x24 (OEM) 320ml x 24
1530	Chakulza OSH MidoriCola 320ml 1x24(OEM) 320ml x 24

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบจะดึงข้อมูลแบบ 3 ระดับ (Level) คือเริ่มจากดึงรายชื่อโรงงานมาแสดง เมื่อเลือกโรงงานเสร็จจะดึงรายชื่อ Line ID และเมื่อเลือก Line ID ที่ต้องการ ระบบจะเรียก API ชุดที่สามเพื่อดึงรายการสินค้า (SKU) ทั้งหมดที่ผลิตในไลน์นั้นมาแสดงผลในรูปแบบตารางทันที

ผลลัพธ์: ผู้ใช้สามารถดูรายละเอียดสินค้า (รหัส SKU, ชื่อสินค้า, ขนาดบรรจุ) ได้แบบ Real-time ในตารางด้านล่าง และทุกการเลือกข้อมูลที่สำคัญจะมีการส่งสัญญาณแจ้งเตือนไปที่ Dashboard บน Server เพื่อบันทึกประวัติการทำงานครับ

Workshop 5-1

Production Setup & BOM

เลือกข้อมูลให้ครบถ้วนเพื่อดูรายการวัตถุดิบ

Client_id

ระบุ Client ID (1-20)

plant_code

3201

▼

Search

Plant_name

▶ ปทุมธานี

line_id

1

▼

sku_id

320731302 - 100Plus Citrus Can325ml 24NR (Export)

▼

Confirm & Send

Ready

รายการวัตถุดิบ (BOM)

Syrup

Require (BOM)

1.58

ลิตร

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบจะดึงข้อมูลแบบลำดับขั้น (Cascading) ตั้งแต่ระดับโรงงาน ไปยังสายการผลิต (Line) จนถึงระดับสินค้า (SKU) โดยความพิเศษคือเมื่อเลือก SKU แล้ว ระบบจะเรียกข้อมูล BOM (Bill of Materials) หรือรายการวัตถุดิบที่ต้องใช้ในการผลิตสินค้านั้นๆ ออกมาแสดงในรูปแบบการ์ดที่ระบุจำนวนและหน่วยวัดของวัตถุดิบแต่ละชนิด (เช่น Syrup, CO2, ฝา, ฉลาก) อย่างชัดเจน

ผลลัพธ์: ผู้ใช้สามารถตรวจสอบรายการวัตถุดิบที่ต้องเตรียมสำหรับการผลิต SKU นั้นๆ ได้ทันทีผ่านหน้าจอ และทุกขั้นตอนการเลือกข้อมูลจะมีการส่งสัญญาณ Log ไปบันทึกที่ Dashboard บน Server เพื่อให้หัวหน้างานติดตามสถานะการตั้งค่าการผลิตได้แบบ Real-time ครับ

Workshop 5-2

Production Setup & BOM

เลือกข้อมูลให้ครบถ้วนเพื่อดูรายการวัตถุดิบ

Client_id

sq Client ID (1-20)

plant_code

3201

Search

Plant_name

ปทุมธานี

line_id

1

sku_id

320731302 - 100Plus Citrus Can325ml 24NR (Export)

Confirm & Send

Ready

รายการวัตถุดิบ (BOM)

Syrup

bom_syrup

Require (BOM)

1.58

ลิต

Update

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบพัฒนาต่อยอดจากการดึงข้อมูล BOM โดยเพิ่มฟังก์ชันให้ผู้ใช้สามารถ "แก้ไขจำนวนวัตถุดิบ" ได้โดยตรงผ่านหน้าเว็บ เมื่อเลือก SKU แล้ว ระบบจะแสดงการดรากรายการวัตถุดิบพร้อมช่องกรอกข้อมูลที่สามารถพิมพ์ตัวเลขใหม่และกดปุ่ม Update เพื่อส่งค่าไปบันทึกลงในฐานข้อมูลที่ Server ได้ทันทีแบบรายรายการ

ผลลัพธ์: ผู้ใช้สามารถบริหารจัดการข้อมูลสูตรการผลิต (BOM) ได้อย่างรวดเร็ว โดยระบบจะแสดงสถานะการบันทึก (Saved!) ให้ทราบ และทุกครั้งที่มีการกดอัปเดต ระบบจะส่งสัญญาณ Log ไปแจ้งเตือนที่ Dashboard บน Server เพื่อบันทึกว่าวัตถุดิบรายการใดถูกแก้ไขเป็นค่าเท่าใดและโดยใครครับ

Workshop 6

Production Summary

ระบบรวบรวมข้อมูลสรุปผลการผลิตรายวัน

Search PO & Client

Client ID
ID (1-20)

PO Number
301010018854

🔄 โหลดข้อมูล

Production Details

Plant ID
3201

Line ID
3

SKU
310706109

Plan Pack
6598

Qty per Pack
12

Pack per Pallet
70

Production Results

Start Date
16/04/2026, 12:50

Ending Date
16/04/2026, 18:35

Output (Total Bottles)
10000

Total Pack
0

Pallet (รวมาน)
0

Pack (รวมแพ็ค)
0

Bottle (รวมขวด)
0

Material BOM Analysis

Section 4

Syrup

หน่วย: ลิตร

Use	Bom	Loss	% Loss
0	0.00	0.00	0.00 %

CO2

หน่วย: กก.

Use	Bom	Loss	% Loss
0	0.00	0.00	0.00 %

Preform

หน่วย: ขวด

Use	Bom	Loss	% Loss
0	0.00	0.00	0.00 %

ขวด/กระป๋อง

หน่วย: ขวด

Use	Bom	Loss	% Loss
0	0.00	0.00	0.00 %

ฝา

หน่วย: ฝา

Use	Bom	Loss	% Loss
0	0.00	0.00	0.00 %

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบจะดึงข้อมูลเลขที่ใบสั่งผลิต (PO) จาก Server มาให้เลือกใน Dropdown เมื่อโหลดข้อมูลแล้ว ระบบจะแสดงรายละเอียดการผลิตทั้งหมด (โรงงาน, ไลน์, สินค้า, แผนการบรรจุ) ความโดดเด่นคือระบบ **วิเคราะห์วัสดุอัตโนมัติ** โดยเมื่อผู้ใช้กรอกจำนวนผลผลิตที่ทำได้จริง (Output) ระบบจะคำนวณจำนวนวัตถุดิบ 16 รายการที่ควรใช้ตามมาตรฐาน (BOM) ทันที และให้ผู้ใช้งานกรอกจำนวนที่ใช้จริงเพื่อเปรียบเทียบหาค่าความสูญเสีย (Loss) และเปอร์เซ็นต์ความสูญเสีย (% Loss) ในรูปแบบการ์ด

ผลลัพธ์: ผู้ใช้จะได้รายงานสรุปการผลิตที่สมบูรณ์ ทั้งจำนวนพาเลท/เศษแพ็ค/เศษขวด และข้อมูลการใช้วัตถุดิบอย่างละเอียด เมื่อกดบันทึก ข้อมูลทั้งหมดจะถูกส่งไปเก็บในฐานข้อมูลหลัก และส่งสัญญาณแจ้งสถานะ "Finished PO" ไปยัง Dashboard ของหัวหน้างานเพื่อตรวจสอบประสิทธิภาพการผลิตในวันนั้นๆ ครับ

Workshop 7

Production Report

ประวัติการบันทึกงานการผลิตทั้งหมด

Export Excel

Search Filters

Section 1

Client ID

Plant ID

Line ID

Production Date

ID (1-20)

3201

3

2026-04-16

Search

Production Table

1 records found

Section 2

DATE	PO NUMBER	PLANT/LINE	BOTTLE OUTPUT	BOM USE (AVG)	LOSS (AVG)	ACTIONS
2026-04-16	301010018854	undefined / 3	1,000	2418.00	2318.00	Edit Remove

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบจะดึงข้อมูลตัวกรอง (Filter) เช่น รายชื่อโรงงาน, ไลน์ผลิต และวันที่ที่มีการบันทึกจริงมาจากฐานข้อมูลมาใส่ใน Dropdown โดยอัตโนมัติ เมื่อผู้ใช้เลือกเงื่อนไขและกด **Search** ระบบจะเรียก API เพื่อดึงรายการสรุปผลการผลิตทั้งหมดมาแสดงในรูปแบบตารางที่ดูง่าย พร้อมความสามารถในการส่งออกข้อมูลเป็นไฟล์ **Excel (CSV)** และมีปุ่ม Action สำหรับเชื่อมโยงไปยังการแก้ไขหรือลบข้อมูลในขั้นตอนถัดไป

ผลลัพธ์: ผู้ใช้สามารถติดตามและตรวจสอบย้อนหลังได้ว่าในแต่ละวันมี Output เท่าไหร่ และมีการใช้วัตถุดิบ (BOM) เป็นอย่างไร โดยระบบจะแสดงจำนวนรายการที่พบ (Record count) และส่งสัญญาณการเข้าดูรายงาน (Search Report) ไปยัง Dashboard ของหัวหน้างานเพื่อบันทึกประวัติการตรวจสอบข้อมูลครับ

Workshop 8

Edit / Remove Report

← Back to Report

Basic Information

Section 1

PO Number

301010018854

Client ID

1

Plant ID

3201

Line ID

3

Date

16/04/2026

Bottle Output

1000

Material Consumption (BOM)

Section 2

Material Description	Bom (Standard)	Use (Actual)	Loss	% Loss
1. Syrup (Litre)	0	2418	2418.00	0.00
2. CO2 (Kg)	0	130	130.00	0.00
3. Preform (Pcs)	0	0	0.00	0.00
4. BT 1500ml (Pcs)	0	12000	12000.0	0.00
5. Cap (Pcs)	0	12000	12000.0	0.00
6. Label (Pcs)	0	11999	11999.00	0.00
7. Glue Label (Kg)	0	0	0.00	0.00
8. Glue Tray (Kg)	0	0	0.00	0.00
9. Tray (Pcs)	0	0	0.00	0.00
10. Barcode PK (Pcs)	0	997	997.00	0.00
11. Barcode PL (Pcs)	0	0	0.00	0.00
12. Film Step 1 (Kg)	0	24.2	24.20	0.00
13. Film Step 2 (Kg)	0	0	0.00	0.00
14. Sheet Top (Pcs)	0	712	712.0	0.00
15. Sheet (Pcs)	0	0	0.00	0.00
16. Wrap (Mtr)	0	0	0.00	0.00

Actions

Control

Notify Client ID:

0

Update Report

Delete Report

แหล่งที่มาของโค้ด :

คำอธิบาย: ระบบจะดึงข้อมูลการผลิตเดิมจากฐานข้อมูลตาม ID ที่ระบุมาแสดงในฟอร์ม โดยแบ่งเป็น 2 ส่วนหลักคือ **ข้อมูลพื้นฐาน** (PO, Plant, Line, Output) และ **ตารางการใช้วัตถุดิบ (BOM)** ทั้ง 16 รายการ ความพิเศษคือในตาราง BOM ระบบจะดึงค่ามาตรฐาน (Standard) มาเปรียบเทียบกับค่าที่ใช้จริง (Actual Use) และคำนวณค่าความสูญเสีย (Loss/% Loss) ให้แบบ Real-time เมื่อมีการแก้ไขตัวเลข

ผลลัพธ์: ผู้ใช้สามารถปรับปรุงข้อมูลการผลิตให้ถูกต้องหรือลบรายงานที่ไม่ต้องการทิ้งได้ โดยเมื่อดำเนินการสำเร็จ ระบบจะแสดงจำนวนแถวที่ได้รับผลกระทบ (Affected rows) และส่งสัญญาณแจ้งเตือน "Updated/Deleted Report" ไปยัง Dashboard ของหัวหน้างานตาม Client ID ที่ระบุไว้ในส่วน Notify เพื่อให้รับทราบการเปลี่ยนแปลงข้อมูลครับ